STOCK FORECASTING DASHBOARD

🔷 SYSTEM ARCHITECTURE & DESIGN

1. Can you explain the overall architecture of your stock forecasting system?

Answer:
The system follows a **client–server architecture**.

- **Frontend (React / NiceGUI)** handles user interaction, input validation, and visualization (charts, risk scores).
- **Backend (Flask)** exposes REST APIs for prediction and risk metrics.
- **ML layer** consists of pre-trained LSTM models per company, scalers, and historical datasets.
  The frontend sends HTTP requests to Flask, which performs inference and returns JSON responses.

---

2. Why did you separate the frontend and backend instead of using a monolithic app?

Answer:
Separation improves **scalability, maintainability, and flexibility**.
Frontend and backend can evolve independently, allows easier API testing, supports multiple clients (web, mobile), and isolates ML inference from UI logic.

---

3. Why Flask instead of Django or FastAPI?

Answer:
Flask is lightweight and ideal for **ML inference services**.
It has minimal overhead, easy integration with TensorFlow, and faster iteration for experimental models compared to Django.
FastAPI was an option, but Flask's ecosystem and simplicity suited this project.

🔹 MACHINE LEARNING & MODELING

4. Why did you choose LSTM for stock price prediction?

Answer:
LSTM is designed for **time-series data** and can capture long-term dependencies in sequential price movements.
Traditional models like ARIMA struggle with non-linear trends, while LSTMs learn complex temporal patterns.

---

5. Why do you use separate models for each company?

Answer:
Each stock has **different volatility, trends, and scale**.
Training separate models improves accuracy and avoids bias that a single generalized model might introduce.

---

6. What is the role of scaling in your pipeline?

Answer:
Scaling ensures stable training and inference by normalizing values.
LSTM networks are sensitive to magnitude; using a scaler like MinMaxScaler prevents exploding gradients and improves convergence.

---

7. Why do you pad predictions before inverse transforming?

Answer:
The scaler expects the **same feature dimensionality** it was trained on.
Since predictions may include only one feature, padding with zeros preserves shape consistency for inverse transformation.

---

8. How does your multi-day forecast work?

Answer:
It uses **recursive forecasting**:

- Predict the next value
- Append it to the input sequence
- Slide the window forward
- Repeat for N days
  This mimics real-world forecasting but may accumulate error over time.

🔷 BACKEND (FLASK & APIs)

9. How do you handle invalid company names?

Answer:
The backend validates company names against predefined mappings.
It supports **case-insensitive matching** and returns structured error messages for invalid input.

---

10. Why do you load models dynamically instead of at startup?

Answer:
Dynamic loading reduces memory usage and allows selective inference.
Only the requested company's model is loaded, which improves scalability.

---

11. How do you handle CORS issues?

Answer:
Flask-CORS is enabled to allow cross-origin requests from the frontend running on a different port.
Without it, browsers would block API calls.

---

12. What happens if required files are missing?

Answer:
The backend checks file existence before inference and returns descriptive errors.
This prevents runtime crashes and improves debuggability.

🔷 GEO-RISK (GRSI) LOGIC

13. What is the GeoRisk Signal Index (GRSI)?

Answer:
GRSI is a composite risk score derived from geopolitical and country-level indicators.
Higher values indicate higher market uncertainty and potential volatility.

---

14. Why did you separate company-level and country-level GRSI?

Answer:
Company-specific risk captures sector and operational exposure, while country GRSI captures macroeconomic and geopolitical risk.
This separation improves interpretability.

---

15. How do you handle missing GRSI data?

Answer:
The backend validates presence of values and returns meaningful errors if missing.
The frontend displays fallback messages instead of failing silently.

🔷 FRONTEND (REACT / NICEGUI)

16. How do you manage state in the frontend?

Answer:
State variables track selected country, company, forecast days, prediction results, and GRSI.
UI re-renders based on state changes to ensure consistency.

---

17. Why did you move from React to NiceGUI?

Answer:
NiceGUI allows **Python-only UI development**, faster prototyping, and tighter backend integration, especially for ML-centric dashboards.

---

18. How do you visualize predictions?

Answer:
Two visualizations are used:

- Interactive line charts for forecasted values
- Static Matplotlib plots for actual vs predicted prices
  This provides both interactivity and model-validation clarity.

---

19. Why did you embed GRSI inside the button instead of a separate card?

Answer:
It improves UX by keeping action and result context together.
The user immediately sees the risk score associated with their action.

🔷 PERFORMANCE & RELIABILITY

20. How do you prevent UI blocking during API calls?

Answer:
NiceGUI uses `run.io_bound()` to offload blocking I/O operations to a background thread, keeping the UI responsive.

---

21. What are the limitations of your system?

Answer:

- Recursive forecasts accumulate error
- No real-time market data
- Models require periodic retraining
- External geopolitical data is static

---

22. How would you improve this system?

Answer:

- Use real-time APIs
- Add ensemble models
- Introduce confidence intervals
- Move inference to async FastAPI
- Add caching (Redis)
- Automate retraining pipelines

🔷 DEPLOYMENT & SECURITY

23. How would you deploy this project?

Answer:

- Dockerize Flask backend
- Use Nginx as reverse proxy
- Serve frontend via CDN

- Deploy on AWS/GCP with autoscaling

---

24. How do you secure the API?

Answer:

- Input validation
- Rate limiting
- Authentication tokens
- HTTPS
- Environment-based configuration

🔷 ADVANCED / TRICK QUESTIONS

25. Why is stock prediction inherently difficult?

Answer:
Markets are non-stationary, influenced by external events, sentiment, and randomness.
Models can detect patterns but **cannot predict black-swan events** reliably.

---

26. How would you evaluate your model's performance?

Answer:
Using RMSE, MAE, directional accuracy, and visual inspection of prediction vs actual trends.

---

27. Why not use Transformers instead of LSTM?

Answer:
Transformers require more data and computational resources.
For limited historical datasets, LSTM offers a better accuracy–complexity trade-off.

REACT ARCHITECTURE & DESIGN

1. How is your React frontend structured for this stock project?

Answer:
The frontend is component-based:

- **Input components** for country, company, and forecast days
- **Service layer** for API calls (Axios)
- **Visualization components** for charts and risk indicators
  State is lifted to parent components to ensure data consistency across the UI.

---

2. Why did you separate API calls into a service layer?

Answer:
It improves **separation of concerns**, reusability, and testability.
If the backend changes, only the service layer needs updates, not UI components.

---

3. How does React communicate with the Flask backend?

Answer:
Using **REST APIs over HTTP**, primarily with Axios.
JSON is used for request and response payloads, enabling clean decoupling.

🔹 STATE MANAGEMENT

4. How do you manage state in your application?

Answer:
I use **React hooks** (`useState`, `useEffect`).
State variables track selected inputs, prediction results, loading states, and GRSI scores.

---

5. Why didn't you use Redux or Context API?

Answer:
The app has **localized state**, not deeply nested global state.
Redux would add unnecessary complexity for a single-dashboard app.

---

6. How do you prevent unnecessary re-renders?

Answer:

- Split UI into smaller components
- Pass only required props
- Use dependency arrays in `useEffect`
- Avoid anonymous functions in JSX when possible

🔹 API HANDLING & ASYNC LOGIC

7. How do you handle asynchronous API calls?

Answer:
Using `async/await` inside event handlers and `useEffect`.
Loading states ensure users receive visual feedback during network operations.

---

8. How do you handle API errors gracefully?

Answer:

- Catch Axios errors
- Display user-friendly error messages
- Prevent UI crashes by validating responses before rendering

---

9. Why did Axios work better than Fetch in your project?

Answer:
Axios automatically rejects non-2xx responses, simplifies JSON handling, and supports interceptors for logging and authentication.

🔹 COMPONENT COMMUNICATION

10. How do you pass prediction data to chart components?

Answer:
The parent component stores prediction results in state and passes them as props to chart components, ensuring unidirectional data flow.

---

11. How do you handle dependent dropdowns (Country → Company)?

Answer:
When the country changes, company state resets.
This avoids stale selections and ensures valid input combinations.

🔹 DATA VISUALIZATION

12. How did you implement the actual vs predicted price graph?

Answer:
Using a charting library (Chart.js / Recharts).

The graph overlays historical actual prices with predicted values for intuitive comparison.

---

13. Why did you choose client-side rendering for charts?

Answer:
Client-side rendering improves responsiveness and avoids unnecessary backend load for visualization logic.

---

14. How do you handle large datasets in charts?

Answer:

- Limit visible points
- Use pagination or zoom
- Avoid unnecessary re-renders
  This ensures smooth performance.

🔷 UX & UI DESIGN

15. How did you improve user experience in your dashboard?

Answer:

- Disabled buttons during API calls
- Loading indicators
- Inline error messages
- Clear labels and visual hierarchy

---

16. Why did you embed GRSI output close to the trigger button?

Answer:
It reduces cognitive load by showing results immediately where the user expects feedback.

---

17. How do you ensure accessibility?

Answer:

- Semantic HTML
- Clear labels
- Keyboard-friendly inputs
- Sufficient contrast

🔷 PERFORMANCE OPTIMIZATION

18. How do you optimize API calls?

Answer:

- Avoid redundant calls
- Cache previous responses
- Trigger calls only on valid user actions

---

19. How do you handle slow backend responses?

Answer:
Loading spinners and timeouts prevent freezing.
Error states ensure graceful degradation.

🔹 SECURITY & RELIABILITY

20. How do you prevent invalid user input?

Answer:
Using controlled components, dropdown constraints, and client-side validation before API calls.

---

21. How do you handle environment-specific configs?

Answer:
Using `.env` files with `REACT_APP_API_BASE_URL` for different environments.

🔹 ADVANCED / SCENARIO QUESTIONS

22. What happens if the backend API schema changes?

Answer:
The service layer isolates the impact.
Only response parsing logic needs adjustment.

---

23. How would you convert this app to SSR (Next.js)?

Answer:

- Move API calls to `getServerSideProps`
- Hydrate charts on the client
- Improve SEO and performance

---

24. How would you test this React frontend?

Answer:

- Unit tests with Jest
- Component tests with React Testing Library
- API mocking using MSW

---

25. What are common React mistakes you avoided?

Answer:

- Direct state mutation
- Excessive re-renders
- Tight coupling between UI and API logic